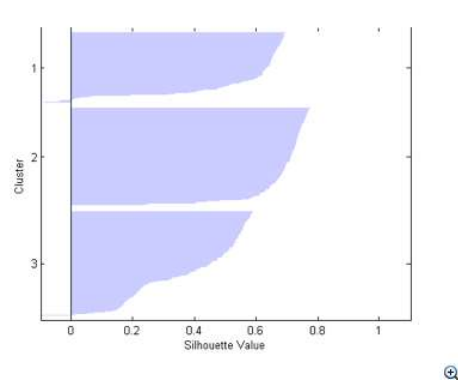


Clustering III



Rachael Hageman Blair

K-medoids

Algorithm 14.1 *K*-means Clustering.

1. For a given cluster assignment C , the total cluster variance (14.33) is minimized with respect to $\{m_1, \dots, m_K\}$ yielding the means of the currently assigned clusters (14.32).
2. Given a current set of means $\{m_1, \dots, m_K\}$, (14.33) is minimized by assigning each observation to the closest (current) cluster mean. That is,

$$C(i) = \operatorname{argmin}_{1 \leq k \leq K} \|x_i - m_k\|^2. \quad (14.34)$$

3. Steps 1 and 2 are iterated until the assignments do not change.
-

We can relax these assumptions.

Distance assumption

K-medoids

Algorithm 14.2 *K-medoids Clustering.*

1. For a given cluster assignment C find the observation in the cluster minimizing total distance to other points in that cluster:

$$i_k^* = \operatorname{argmin}_{\{i: C(i)=k\}} \sum_{C(i')=k} D(x_i, x_{i'}). \quad (14.35)$$

Then $m_k = x_{i_k^*}$, $k = 1, 2, \dots, K$ are the current estimates of the cluster centers.

2. Given a current set of cluster centers $\{m_1, \dots, m_K\}$, minimize the total error by assigning each observation to the closest (current) cluster center:

$$C(i) = \operatorname{argmin}_{1 \leq k \leq K} D(x_i, m_k). \quad (14.36)$$

3. Iterate steps 1 and 2 until the assignments do not change.

Example: from book

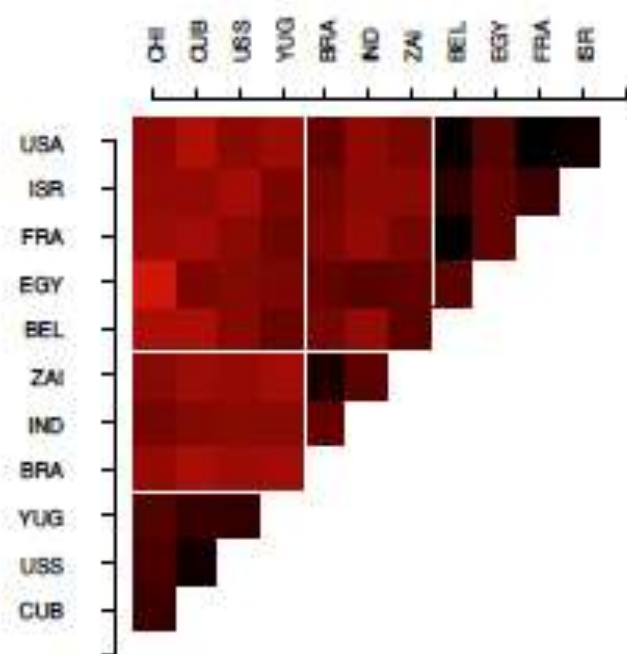
The Data

- Similarity measures between 12 Countries:
Belgium, Brazil, Chile, Cuba, Egypt, France, India, Israel, United States, Union of Soviet Socialist Republics, Yugoslavia and Zaire.
- Based on Student opinion.
- K means will not work.
- K-medoid clustering applied with $K=3$.

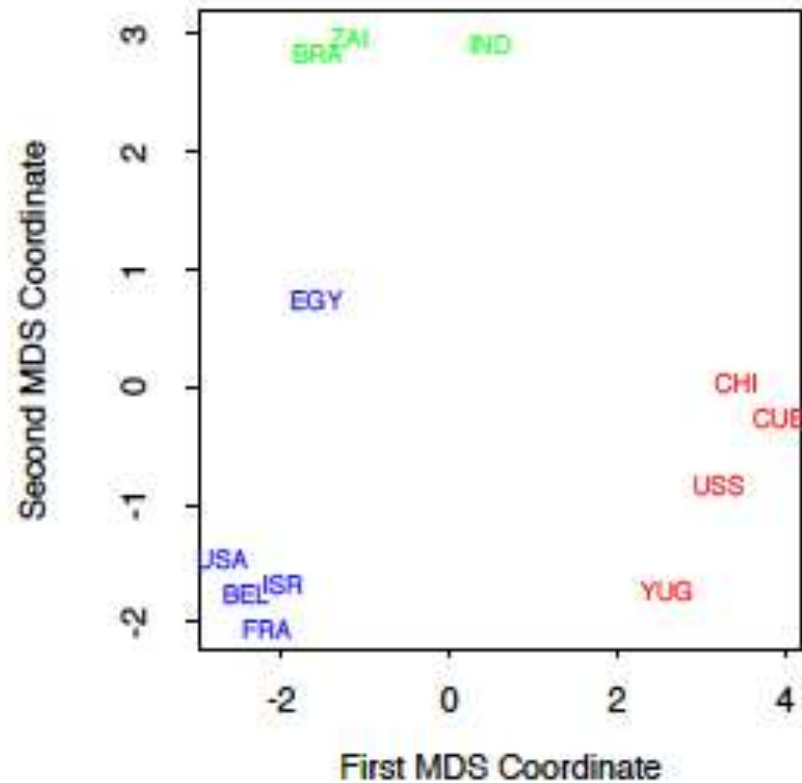
Example

	BEL	BRA	CHI	CUB	EGY	FRA	IND	ISR	USA	USS	YUG
BRA	5.58										
CHI	7.00	6.50									
CUB	7.08	7.00	3.83								
EGY	4.83	5.08	8.17	5.83							
FRA	2.17	5.75	6.67	6.92	4.92						
IND	6.42	5.00	5.58	6.00	4.67	6.42					
ISR	3.42	5.50	6.42	6.42	5.00	3.92	6.17				
USA	2.50	4.92	6.25	7.33	4.50	2.25	6.33	2.75			
USS	6.08	6.67	4.25	2.67	6.00	6.17	6.17	6.92	6.17		
YUG	5.25	6.83	4.50	3.75	5.75	5.42	6.08	5.83	6.67	3.67	
ZAI	4.75	3.00	6.08	6.67	5.00	5.58	4.83	6.17	5.67	6.50	6.92

Example



Reordered Dissimilarity Matrix



How to choose k?

- Sometimes the data dictates what k should be.
- Heuristic approaches, examine the within cluster dissimilarity W_K as a function of the number of classes $K \in \{1, 2, \dots, K_{\max}\}$.
- Within cluster variation typically decreases as a function of K . Cross-validation cannot help us.
- Comforting because we can assume that if there are K^* clusters, and if K is “true”, and $K < K^*$, then the algorithm will return groups that are actually subsets of the true underlying groups.
Does not necessarily hold up in practice.

Silhouettes: a graphical aid to the interpretation and validation of cluster analysis

Peter J. ROUSSEEUW

University of Fribourg, ISES, CH-1700 Fribourg, Switzerland

Received 13 June 1986

Revised 27 November 1986

Abstract: A new graphical display is proposed for partitioning techniques. Each cluster is represented by a so-called *silhouette*, which is based on the comparison of its tightness and separation. This silhouette shows which objects lie well within their cluster, and which ones are merely somewhere in between clusters. The entire clustering is displayed by combining the silhouettes into a single plot, allowing an appreciation of the relative quality of the clusters and an overview of the data configuration. The average silhouette width provides an evaluation of clustering validity, and might be used to select an ‘appropriate’ number of clusters.

Keywords: Graphical display, cluster analysis, clustering validity, classification.

Let us first define the numbers $s(i)$ in the case of dissimilarities. Take any object i in the data set, and denote by A the cluster to which it has been assigned. (For a concrete illustration, see Fig. 1). When cluster A contains other objects apart from i , then we can compute

$$a(i) = \text{average dissimilarity of } i \text{ to all other objects of } A.$$

In Fig. 1, this is the average length of all lines within A . Let us now consider any cluster C which is different from A , and compute

$$d(i, C) = \text{average dissimilarity of } i \text{ to all objects of } C.$$

In Fig. 1, this is the average length of all lines going from i to C . After computing $d(i, C)$ for all clusters $C \neq A$, we select the smallest of those numbers and denote it by

$$b(i) = \min_{C \neq A} d(i, C).$$

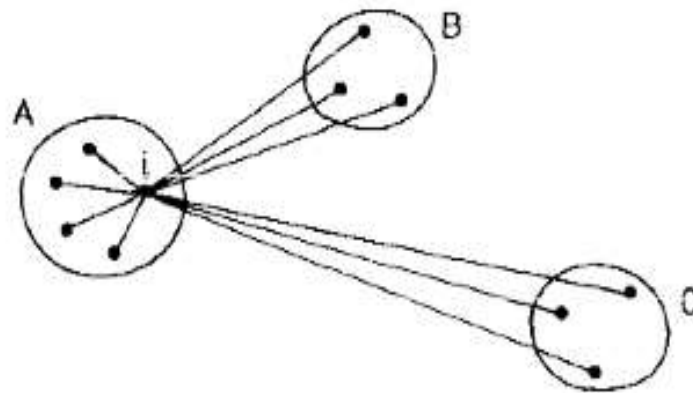


Fig. 1. An illustration of the elements involved in the computation of $s(i)$, where the object i belongs to cluster A .

The number $s(i)$ is obtained by combining $a(i)$ and $b(i)$ as follows:

$$s(i) = \begin{cases} 1 - a(i)/b(i) & \text{if } a(i) < b(i), \\ 0 & \text{if } a(i) = b(i), \\ b(i)/a(i) - 1 & \text{if } a(i) > b(i). \end{cases}$$

It is even possible to write this in one formula:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

Silhouette Plots

- Each cluster is represented by a so-called *silhouette*, which is based on the comparison of its tightness and separation.
- This silhouette shows which objects lie well within their cluster, and which ones are merely somewhere in between clusters.

Range of SC	Interpretation
0.71-1.0	A strong structure has been found
0.51-0.70	A reasonable structure has been found
0.26-0.50	The structure is weak and could be artificial
< 0.25	No substantial structure has been found

Silhouette Plots

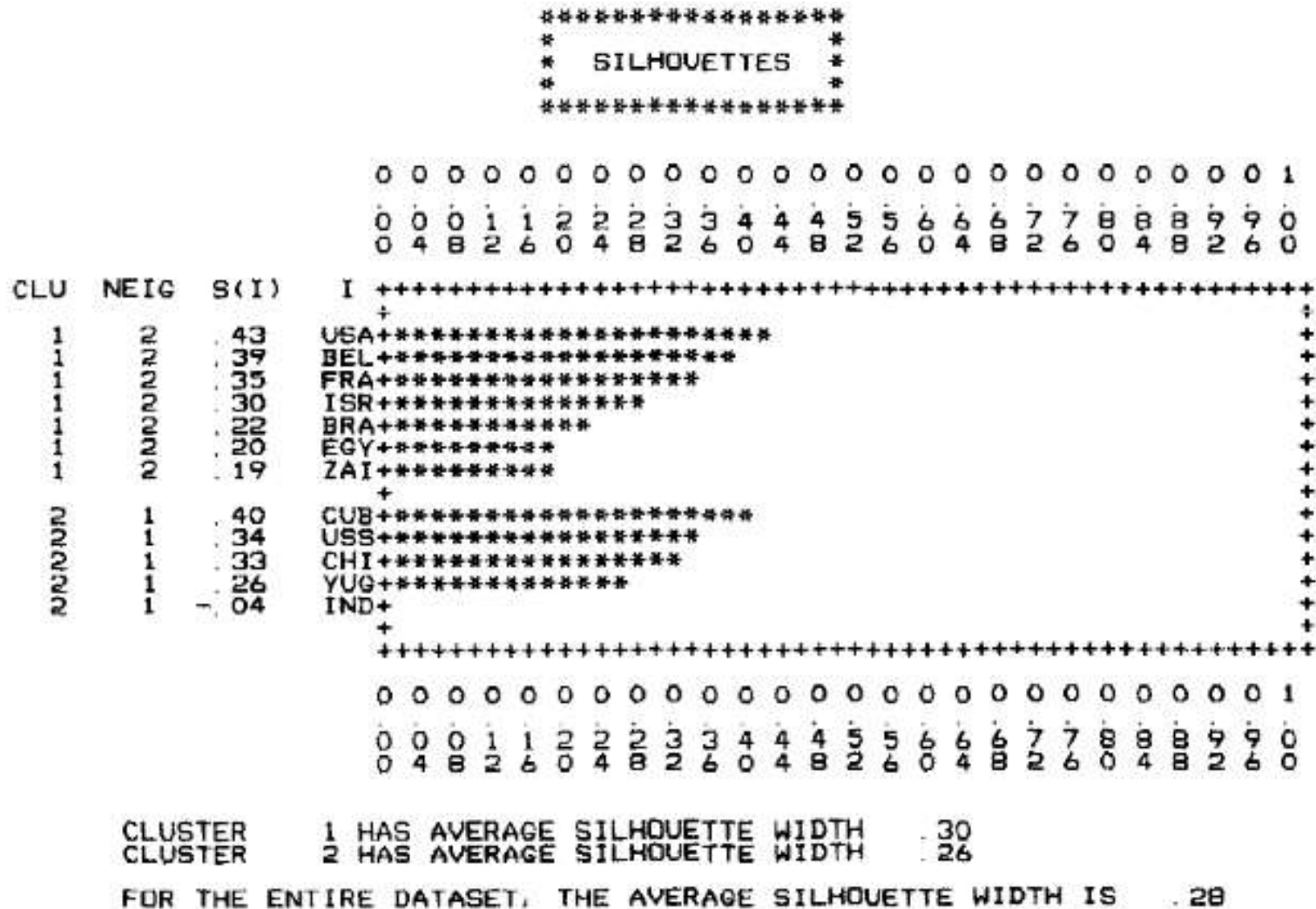


Fig. 2. Silhouettes of a clustering with $k = 2$ of the twelve countries data of Table 1.

Silhouette Plots

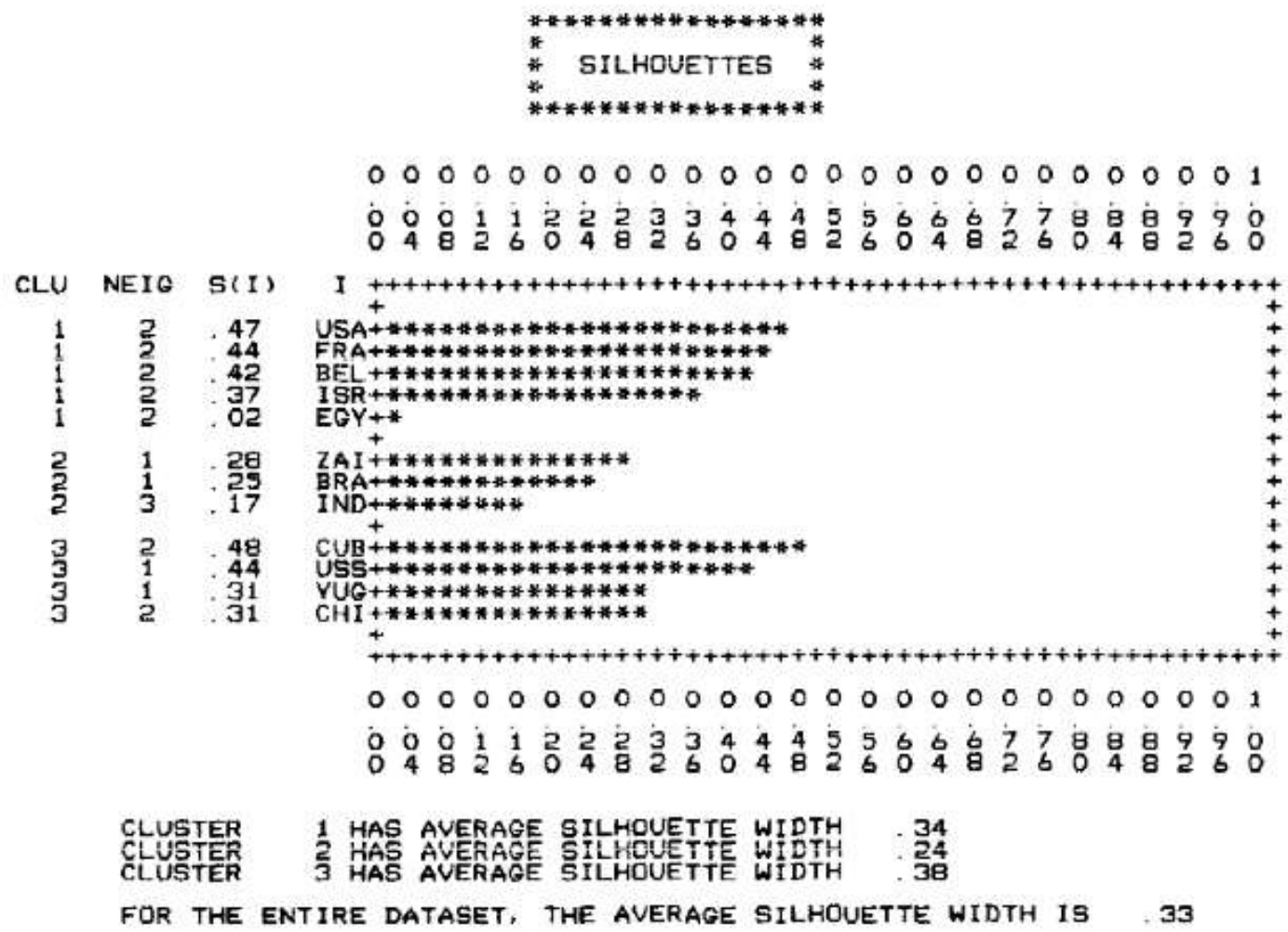


Fig. 3. Silhouettes of a clustering with $k = 3$ of the twelve countries data.

Silhouette Plots

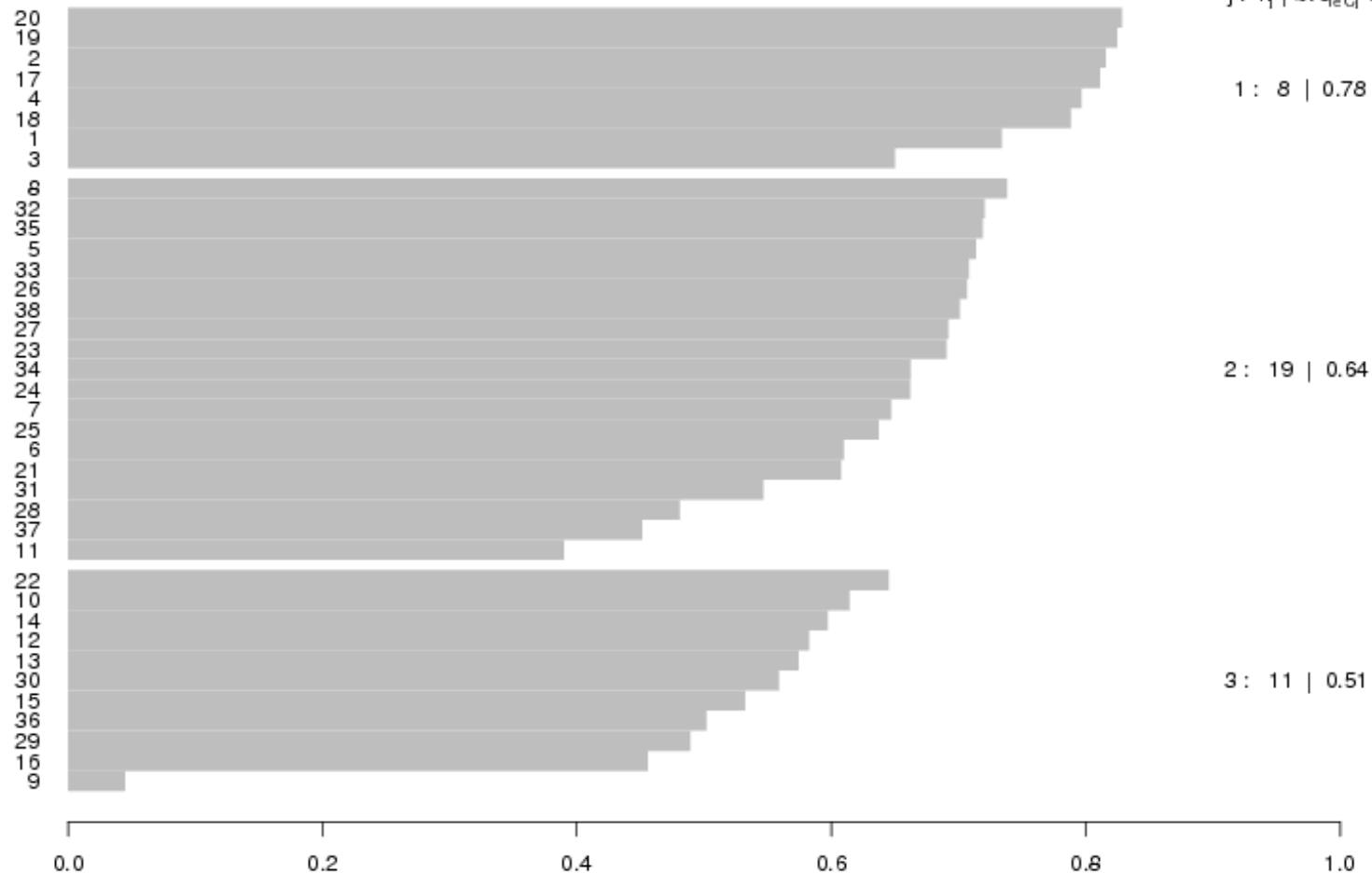
```
> plot(cars.pam)
```

Silhouette plot of pam(x = cars.dist, k = 3)

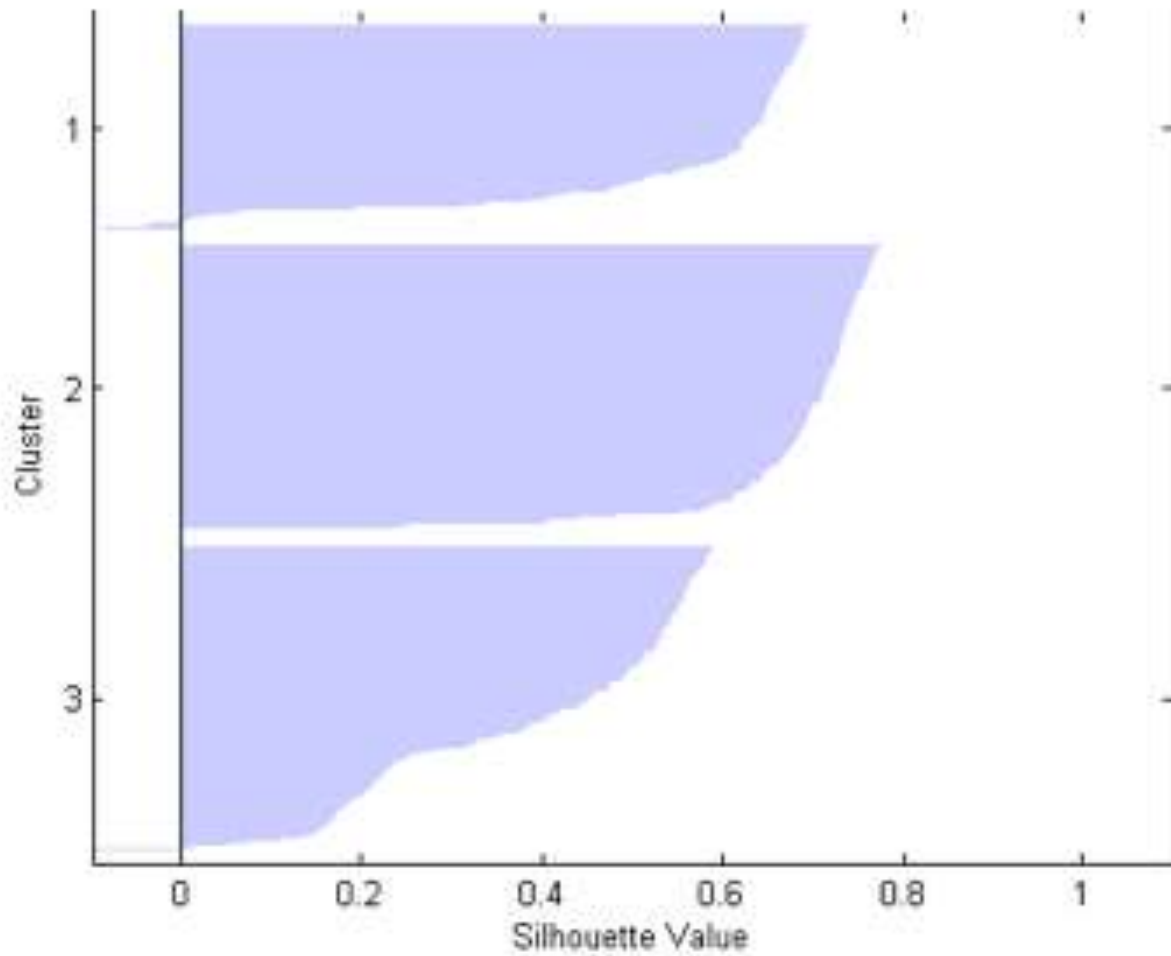
n = 38

3 clusters C_j

$j: n_j | \text{ave}_{i \in C_j} s_i$



Silhouette Plots



Estimating the number of clusters in a data set via the gap statistic

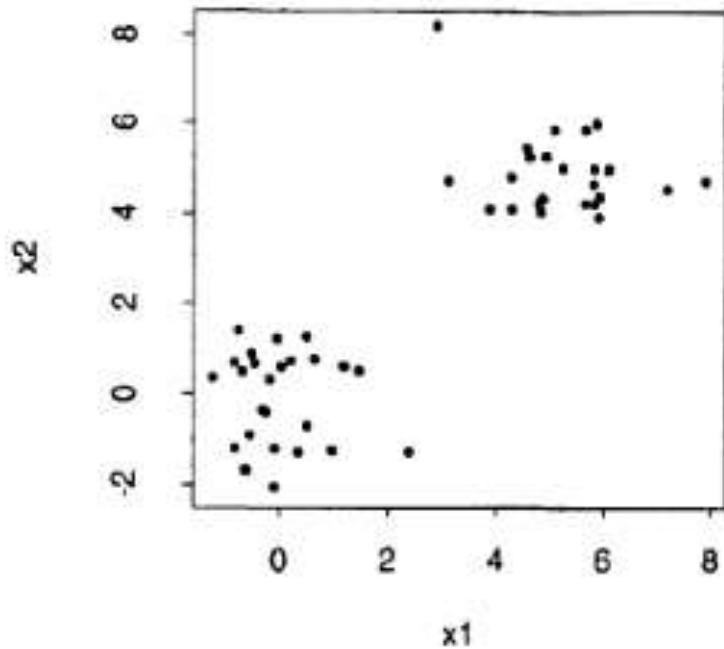
Robert Tibshirani, Guenther Walther and Trevor Hastie
Stanford University, USA

[Received February 2000. Final revision November 2000]

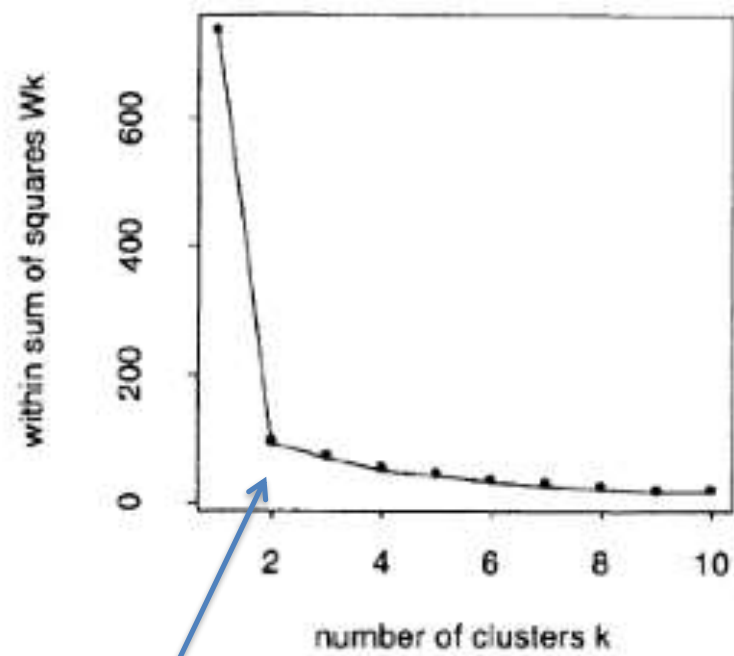
Summary. We propose a method (the ‘gap statistic’) for estimating the number of clusters (groups) in a set of data. The technique uses the output of any clustering algorithm (e.g. *K*-means or hierarchical), comparing the change in within-cluster dispersion with that expected under an appropriate reference null distribution. Some theory is developed for the proposal and a simulation study shows that the gap statistic usually outperforms other methods that have been proposed in the literature.

Keywords: Clustering; Groups; Hierarchy; *K*-means; Uniform distribution

Rule of Thumb...



(a)



(b)

Hunt for the “elbow”.

Gap Statistic

- Compares the curve $\log(W_k)$ to the curve obtained from data uniformly distributed over a rectangle containing the data.
- Estimates the optimal number of clusters as the place where the gap between the two curves is the largest. Works well, and even when there is only one cluster.

Gap Statistic

- Suppose we have clustered the data into k clusters, $C_1, C_2, \dots, C_k$, with $n_r = |C_r|$, as the number of observations in the cluster.
- Let $D_r = \sum_{i, i' \in C_r} d_{ii'}$, be the sum of pairwise distances for all points in

cluster r , and set:

$$W_k = \sum_{r=1}^k \frac{1}{2n_r} D_r.$$

** The idea is similar to comparing against a reference distribution, in this case, the reference is the null.*

Gap Statistic

- Gap Statistic is defined as:

$$Gap_n(k) = E_n^* \{\log(W_k)\} - \log(W_k),$$

where E_n^* denotes the expectation under a sample size n from the reference distribution.

- The estimate $\hat{k}$ will be the estimate of the value maximizing $Gap_n(k)$ after the reference distribution is taken into account.

****Note** this is a general estimate and applicable to any clustering method that relies on determination of k .

Gap Statistic

We consider two choices for the reference distribution:

- (a) generate each reference feature uniformly over the range of the observed values for that feature;
- (b) generate the reference features from a uniform distribution over a box aligned with the principal components of the data. In detail, if X is our $n \times p$ data matrix, assume that the columns have mean 0 and compute the singular value decomposition $X = UDV^T$. We transform via $X' = XV$ and then draw uniform features Z' over the ranges of the columns of X' , as in method (a) above. Finally we back-transform via $Z = Z'V^T$ to give reference data Z .

Method (a) has the advantage of simplicity. Method (b) takes into account the shape of the data distribution and makes the procedure rotationally invariant, as long as the clustering method itself is invariant.

Gap Statistic

Computation of the gap statistic proceeds as follows.

Step 1: cluster the observed data, varying the total number of clusters from $k = 1, 2, \dots, K$, giving within-dispersion measures $W_k, k = 1, 2, \dots, K$.

Step 2: generate B reference data sets, using the uniform prescription (a) or (b) above, and cluster each one giving within-dispersion measures $W_{kb}^*, b = 1, 2, \dots, B, k = 1, 2, \dots, K$. Compute the (estimated) gap statistic

$$\text{Gap}(k) = (1/B) \sum_b \log(W_{kb}^*) - \log(W_k).$$

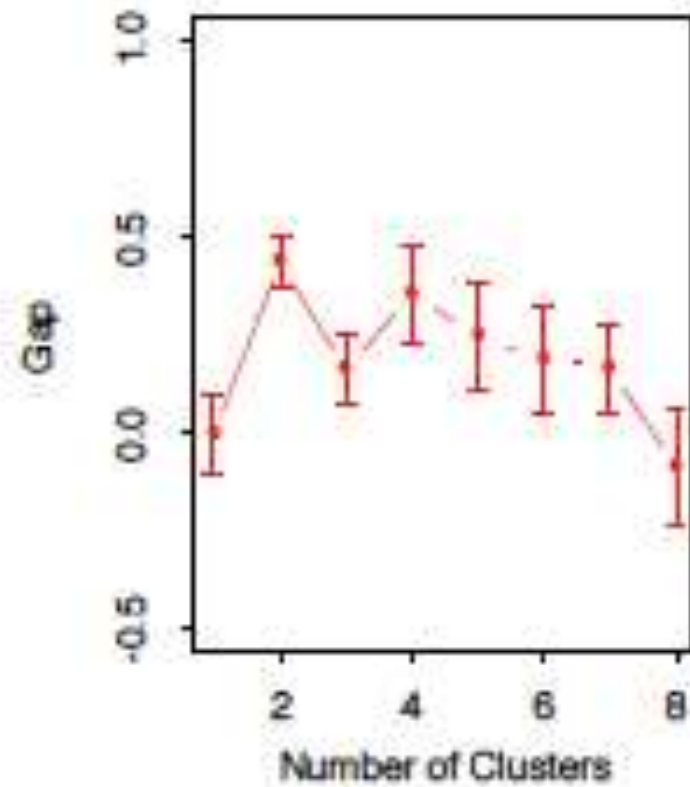
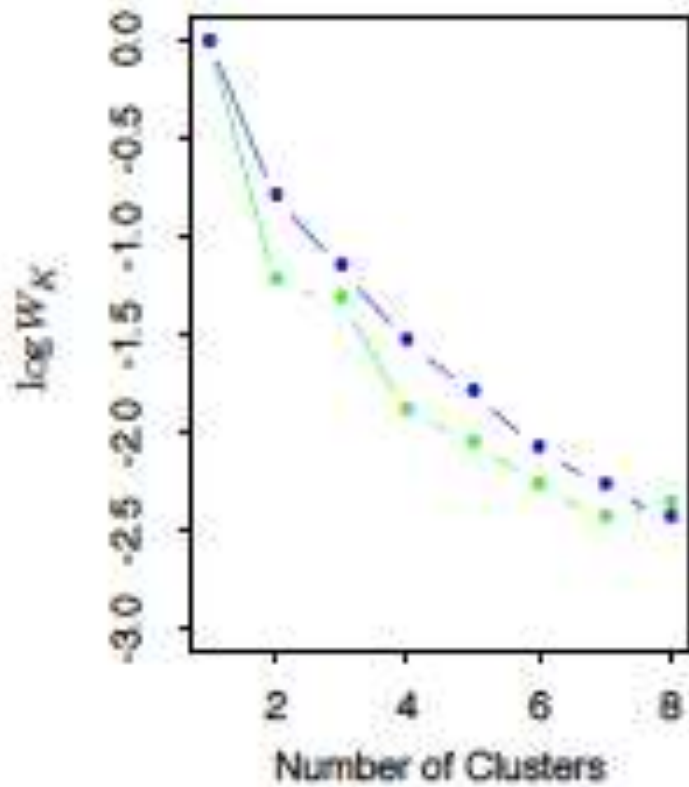
Step 3: let $\bar{l} = (1/B) \sum_b \log(W_{kb}^*)$, compute the standard deviation

$$\text{sd}_k = [(1/B) \sum_b \{\log(W_{kb}^*) - \bar{l}\}^2]^{1/2}$$

and define $s_k = \text{sd}_k \sqrt{1 + 1/B}$. Finally choose the number of clusters via

$$\hat{k} = \text{smallest } k \text{ such that } \text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}.$$

Gap Statistic



Gap Statistics

R Documentation

Gap Statistic for Estimating the Number of Clusters

Description

`clusGap()` calculates a goodness of clustering measure, the “gap” statistic. For each number of clusters k , it compares $\log(W(k))$ with $E^*[\log(W(k))]$ where the latter is defined via bootstrapping, i.e. simulating from a reference distribution.

`maxSE(f, SE.f)` determines the location of the **maximum** of f , taking a “1-SE rule” into account for the *SE* methods. The default method “`firstSEmax`” looks for the smallest k such that its value $f(k)$ is not more than 1 standard error away from the first local maximum. This is similar but not the same as “`Tibs2001SEmax`”, Tibshirani et al’s recommendation of determining the number of clusters from the gap statistics and their standard deviations.

Usage

```
clusGap(x, FUNcluster, K.max, B = 100, verbose = interactive(), ...)
```

```
maxSE(f, SE.f,
      method = c("firstSEmax", "Tibs2001SEmax", "globalSEmax",
                  "firstmax", "globalmax"),
      SE.factor = 1)
## S3 method for class 'clusGap'
print(x, method = "firstSEmax", SE.factor = 1, ...)
```